Data Structures (ETCCDS202)

Unit–3

Assignment: Sorting Algorithms

Course Details

Name of the School: School of Engineering & Technology

Program: B.Tech (CSE-AI/ML)

Semester: 1 Batch: 2025-29

Course Title: Data Structures

Course Code: ETCCDS202

Student Name: Payal

Roll No.: 2501730224

Section: A

Q1. Implement the Core Sorting Algorithms (Insertion, Merge, Quick Sort). Explain their working.

Answer:

Sorting algorithms are used to arrange data in a specific order (ascending or descending). In this assignment, three important sorting algorithms were implemented: Insertion Sort, Merge Sort, and Quick Sort. Each algorithm works differently and has its own advantages and limitations.

Insertion Sort

Insertion Sort works in a similar way to sorting playing cards in hand. It takes one element at a time and places it at its correct position in the already sorted part of the array. Initially, the first element is considered sorted, and then the next elements are compared and inserted accordingly.

This algorithm is simple and easy to understand. It performs very well when the data is already sorted or nearly sorted. However, for large datasets, it becomes inefficient because it compares each element multiple times.

Merge Sort

Merge Sort follows the **divide and conquer approach**. It divides the array into two halves recursively until each part contains a single element. Then, it merges these parts back together in a sorted manner.

Merge Sort is very efficient and always performs in O(n log n) time, regardless of the input. It is also a stable algorithm, meaning it maintains the relative order of equal elements. However, it requires extra memory to store temporary arrays during merging.

Quick Sort

Quick Sort also uses the divide and conquer method. It selects a pivot element and partitions the array such that elements smaller than the pivot are on the left, and larger ones are on the right.

In this implementation, a **random pivot** was used to avoid worst-case performance. Quick Sort is usually very fast and efficient, but if the pivot is poorly chosen (like always the last element in a sorted array), it can degrade to $O(n^2)$.

Q2. Measure and compare the performance of sorting algorithms on different datasets.

Answer:

To analyze performance, three types of datasets were used:

- Random data
- Already sorted data
- Reverse sorted data

Each dataset was tested for sizes: 1000, 5000, and 10000.

Observations:

- **Random Data:**
  Merge Sort and Quick Sort performed much faster compared to Insertion Sort. This is because they have O(n log n) complexity, while Insertion Sort has $O(n^2)$.
- **Sorted Data:**
  Insertion Sort performed extremely well because its best-case complexity is O(n). Quick Sort slowed down when a bad pivot was chosen.
- **Reverse Sorted Data:**
  Insertion Sort performed the worst because it had to shift elements repeatedly. Quick Sort also showed slower performance due to unbalanced partitioning.

Overall, Merge Sort showed consistent performance across all datasets.

Q3. Explain the difference between $O(n^2)$ and O(n log n) using your results.

Answer:

Time complexity describes how the execution time of an algorithm increases with input size.

- **$O(n^2)$ (Insertion Sort):**
  In this complexity, the number of operations increases quadratically. When the input size doubles, the time increases roughly four times. This makes the algorithm inefficient for large datasets.
- **O(n log n) (Merge Sort & Quick Sort):**
  These algorithms grow much slower compared to $O(n^2)$. Even when the input size increases significantly, the time increase is manageable.

From Results:

- Insertion Sort became very slow for large inputs (5000, 10000).

- Merge Sort and Quick Sort handled large data efficiently.

Thus, O(n log n) algorithms are preferred for large-scale problems.

Q4. Explain worst-case behaviour of Quick Sort.

Answer:

Quick Sort performs efficiently in most cases, but its performance depends heavily on the pivot selection.

Worst Case Scenario:

The worst case occurs when:

- The pivot is always the smallest or largest element
- Input is already sorted or reverse sorted

In such cases, the array is divided very unevenly, leading to:

- Deep recursion
- Increased comparisons
- Time complexity becoming **$O(n^2)$**

In this assignment:

When the last element was used as pivot, the algorithm showed poor performance on sorted data. It even caused a **RecursionError** due to deep recursion.

Solution Applied:

To fix this issue, a **random pivot strategy** was used. This improved performance and prevented worst-case behavior.

Q5. Explain stability and memory usage of sorting algorithms.

Answer:

Stability

A sorting algorithm is called stable if it maintains the relative order of elements with equal values.

- **Insertion Sort:** Stable
- **Merge Sort:** Stable
- **Quick Sort:** Not stable

Memory Usage

- **In-place:** Uses constant extra memory
- **Out-of-place:** Requires additional memory

Algorithm	Stable	In-place
Insertion Sort	Yes	Yes
Merge Sort	Yes	No
Quick Sort	No	Yes

Merge Sort requires extra space for merging, while Insertion and Quick Sort mostly work within the same array.

Q6. Explain time complexity of Stack and Queue operations.

Answer:

Stack Operations

A stack follows the **LIFO (Last In First Out)** principle.

- **Push:** O(1) → Insert element at top
- **Pop:** O(1) → Remove top element
- **Peek:** O(1) → View top element

All operations take constant time because they only involve the top element.

Queue Operations

A queue follows the **FIFO (First In First Out)** principle.

- **Enqueue:** O(1) → Add element at rear
- **Dequeue:** O(1) → Remove front element
- **Front:** O(1) → View first element

These operations are efficient because they do not require shifting elements.

Q7. Explain amortized time complexity of append().

Answer:

In Python lists, the append() operation is generally very fast.

- Most of the time, append works in **O(1)** time
- However, when the list becomes full, Python resizes the array
- Resizing requires copying all elements → takes **O(n)** time

Amortized Analysis

Although some operations are costly, they happen rarely. When we average the cost over many operations, the time complexity becomes:

Amortized Time Complexity = O(1)